Error Log

Noorinder

Credit Name: Chapter 13
Assignment Name: StackList

1. Forgetting to Initialize `top`

- **Error:** The `top` pointer was not initialized in the constructor, causing a `NullPointerException` during stack operations.
- **Cause:** The `top` pointer should be initialized to `null` to represent an empty stack, but this step was missed during initial implementation.
- **Fix:** Initialized `top` to `null` in the `StackList` constructor.

Before Fix:

```
public StackList() {
    // No initialization for top
}
```

After Fix:

```
public StackList() {
    top = null; // Initialize top to null
}
```

2. Missing Link in Push Operation

- **Error:** When adding a new item to the stack, the `next` pointer of the new node was not linked to the current `top`. This broke the stack structure, making previously added elements inaccessible.
- **Cause:** The new node wasn't linked to the existing stack before updating the `top` pointer.
- **Fix:** Added logic to set the `next` pointer of the new node to the current `top` before updating the `top` pointer.

Before Fix:

```
public void push(Object item) {
    Node newNode = new Node(item);
    top = newNode; // Directly assigns newNode to top, breaking the link
}
```

After Fix:

```
public void push(Object item) {
    Node newNode = new Node(item);
    newNode.setNext(top); // Link the new node to the current top
    top = newNode;        // Update the top pointer
}
```

3. Handling Empty Stack in Pop Operation

- **Error:** The `pop()` method didn't check if the stack was empty before attempting to remove an item, leading to runtime errors like `NullPointerException`.
- **Cause:** No condition was added to check for an empty stack before accessing the `top` node.
- **Fix:** Implemented a check using the `isEmpty()` method to validate that the stack is not empty before proceeding with the `pop()` operation.

Before Fix:

```
public Object pop() {
    Object item = top.getData(); // Causes NullPointerException if stack is empty
    top = top.getNext();
    return item;
}
```

After Fix:

```
public Object pop() {
    if (isEmpty()) { // Check if the stack is empty
        System.out.println("Stack is empty. Cannot pop.");
        return null;
    }
    Object item = top.getData();
    top = top.getNext();
    return item;
}
```

4. Inefficient Size Calculation

- **Error:** The `size()` method traversed the entire stack each time it was called, leading to inefficient performance for larger stacks.
- **Cause:** The stack size was calculated dynamically by iterating through all nodes instead of maintaining a running count.
- **Fix:** Added a `size` variable that increments or decrements during `push()` and `pop()` operations, enabling constant-time size retrieval.

Before Fix:

```
public int size() {
    int count = 0;
    Node current = top;
    while (current != null) {
        count++;
        current = current.getNext();
    }
    return count;
}
```

After Fix:

```java
private int size; // Add a size variable

public StackList() {
    top = null;
    size = 0; // Initialize size to 0
}

public void push(Object item) {
    Node newNode = new Node(item);
    newNode.setNext(top);
    top = newNode;
    size++; // Increment size when adding an item
}

public Object pop() {
    if (isEmpty()) {
        System.out.println("Stack is empty. Cannot pop.");
        return null;
    }
    Object item = top.getData();
    top = top.getNext();
    size--; // Decrement size when removing an item
    return item;
}

public int size() {
    return size; // Return the size variable directly
}
```

5. Error Handling in Peek Operation

- **Error:** The `peek()` method didn't check if the stack was empty before accessing the `top` node, leading to runtime exceptions when the stack was empty.
- **Cause:** No condition was added to validate if the stack was empty before attempting to return the top element.
- **Fix:** Implemented a condition to check for an empty stack and return an error message if no items were available.

Before Fix:

```java
public Object peek() {
    return top.getData(); // Causes NullPointerException if stack is empty
}
```

After Fix:

```java
public Object peek() {
    if (isEmpty()) { // Check if the stack is empty
        System.out.println("Stack is empty. No top item.");
        return null;
    }
    return top.getData();
}
```